

Analysis: Book Reading

Test set 1

We can solve this test set by naively computing the number of pages that each lazy readers will read. We can do this by initially having an array `torn` of N booleans, where `torn[x]` is true if and only if page x is torn out, and then for each lazy reader i , we can iterate from 1 to N , incrementing our answer only if the value that we iterate is a multiple of R_i and not torn out.

The running time of this solution is $O(N \times Q)$.

Test set 2

Let $f(x)$ be the number of pages that are multiples of x and not torn out. To compute $f(x)$, we can only check whether the pages $x, 2x, 3x, \dots, \text{floor}(N/x)x$ are torn out. Therefore, we can do this in N/x time.

This means that we can compute $f(1), f(2), \dots, f(N)$ in a total of $N(1/1 + 1/2 + \dots + 1/N)$ time. $1/1 + 1/2 + \dots + 1/N$ is approximately $O(\log N)$ (since the n -th [harmonic number](#) is approximately $O(\log N)$), so in total $f(1), f(2), \dots, f(N)$ can be computed in a total of $O(N \log N)$ time.

After precomputing $f(x)$, we can easily count the number of pages that each lazy readers will read in $O(1)$. The running time of this solution is $O(N \log N + Q)$.

Analysis: Shifts

Test set 1 (Visible)

For each shift, we have three choices,

1. Aninda alone guards the shift
2. Boon-Nam alone guards the shift
3. Both of them guard the shift

We can go through all possible choices for all shifts, and check for how many combinations both of their happiness score is at least H . There are total 3^N such combinations possible, enumerating through all of them fits the time limit for test set 1.

Test set 2 (Hidden)

For test set 2, we can divide the shifts into 2 sets, each having the size of at most $\text{ceil}(N/2)$. We can divide them into two sets, because the choices for each shift are independent of each other. For each set, we can enumerate every possible combination, and compute happiness score for both of the guards for each combination, which gives us a list of happiness score pairs for those two sets. Then for each happiness score pair (h_1, h_2) of the combinations of set 1, we need to count the number of happiness score pairs (p_1, p_2) of the combinations of set 2, such that $p_1 + h_1 \geq H$ and $p_2 + h_2 \geq H$, which can be converted to $p_1 \geq H - h_1$ and $p_2 \geq H - h_2$.

Essentially, we can follow these steps below to find the answer.

1. Store the happiness score pairs of combinations of both set 1 and set 2 in two arrays, let's call them A_1 and A_2 .
2. Sort both A_1 and A_2 , by the first value in the pair in decreasing order, then by the second value of the pair in decreasing order.
3. Iterate through A_1 . For each pair (h_1, h_2) in A_1 , do the following:
 - Find the pairs (p_1, p_2) in A_2 where p_1 is not less $H - h_1$.
 - For each of the pair (p_1, p_2) found, add p_2 to a data structure.
 - Find the number of elements not less than $H - h_2$ in the data structure, add that to answer.

For this approach, we need a data structure that supports these two operations:

1. Insert a number in the data structure
2. Count number of elements in the data structure not less than a given integer

These two operations can be done in $O(\log_2(X))$ time using [range tree](#), where X is the number of elements in the data structure. Here upper bound of X is $3^{(N/2)}$. The total complexity of this approach will be $O(3^{(N/2)} \times \log_2(3^{(N/2)})) \sim O(3^{(N/2)} \times N)$ which is sufficient for test set 2.

Analysis: The Equation

Test set 1 (Visible)

For the first test set, notice that the maximum value of k is 127. This is because each A_i is at most 100, so the leading digit of A_i is at most $2^6 = 64$. If $k \geq 128$, then the leading digit of k is at least $2^7 = 128$, meaning that $(A_i \text{ xor } k) \geq 128 > M$.

Hence, we can compute the answer by checking each value of k less than 128 and finding the largest one which produces a sum less than M .

Test set 2 (Hidden)

For the second test set, the reasoning above tells us that $k < 2^{50}$, which is too big for us to check every value.

Instead, notice that each bit of k only affects a single bit of each A_i . We can use this property to compute each bit of k separately.

For each $1 \leq i \leq 50$, define $ones(i)$ to be the number of rules A_i with the i -th bit (numbered starting from the least significant bit) equal to 1. Likewise, define $zeroes(i)$ to be the number of rules with the i -th bit equal to 0. Then we can re-write the sum:

$$\sum_{1 \leq j \leq N} A_j \text{ xor } k$$

as:

$$\sum_i : i\text{-th bit of } k \text{ is } 1 \ 2^i \times zeroes(i) + \sum_i : i\text{-th bit of } k \text{ is } 0 \ 2^i \times ones(i)$$

Note that we can minimize this sum by choosing the i -th bit of k to be 1 if $ones(i) \geq zeroes(i)$, or 0 otherwise. Define $f(j)$ to be the minimum value of the above sum over all bits $i \leq j$. We can use $f(j)$ to determine if a feasible value of k exists for the lowest j bits, which lets us solve this problem greedily. The greedy solution is as follows: starting from the most significant bit i , check if we can set it to be one (by adding cost of setting this bit to one and $f(i-1)$). If this value is less than or equal to m , there exists a feasible k with the i -th bit set to one. Since we want to set to maximize k , it is optimal for us to set this bit to 1. Otherwise, if the sum is larger than m , set the bit to zero. Then iterate by decreasing m by the cost at the current bit and checking the next most significant bit ($i-1$). In this way, we are able to find the largest feasible k . If $f(i)$ is precomputed, the runtime of this algorithm is $O(N \log(\max(A_i)))$.

Note that since $A_i \leq 10^{15}$ and $N \leq 1000$, the maximum sum is a little more than 10^{18} , so using 64-bit integers is sufficient for this problem.